جزوه درجه یک سیستم عامل

فصل ۴ - بخش ۷: مثال‌های سیستم عاملی (پیاده‌سازی نخ)

تا اینجا، همه چیز تئوری بود. حالا می‌خواهیم ببینیم نخ‌ها در دنیای واقعی، یعنی در هسته **ویندوز** و **لینوکس**، چگونه پیاده‌سازی شده‌اند. این بخش، بخش مقایسه‌ای و بسیار مهمی است.

۴-۷-۱: نخ‌های ویندوز (Windows Threads)

فلسفه اصلی: مدل یک-به-یک ویندوز از مدل "یک-به-یک" (One-to-One) که در بخش ۳ یاد گرفتیم، استفاده می‌کند. این یعنی:

- به ازای هر "نخ سطح کاربر" (User Thread) که شما می‌سازید، "هسته" (Kernel) یک "نخ سطح هسته" (Kernel Thread) متناظر برای آن ایجاد می‌کند.
- مدیریت و زمان‌بندی به طور کامل توسط هسته انجام می‌شود.

ساختمان داده‌های نخ در ویندوز (سوال ۱۰۰٪ امتحانی) هر نخ در ویندوز از ۳ ساختمان داده کلیدی تشکیل شده است که درک آن‌ها حیاتی است:

1. **TEB (Thread Environment Block - بلاک محیط نخ):**
   - **کجا زندگی می‌کند؟** در **فضای کاربر (User Space)**.
   - **چیست؟** این "کارت شناسایی" نخ است. هر نخ می‌تواند TEB خودش را بخواند.
   - **شامل چه چیزهایی است؟** شناسه نخ (Thread ID)، **پشته سطح کاربر (User Stack)** و حافظه محلی نخ (TLS).
1. **ETHREAD (Executive Thread Block - بلاک اجرایی نخ):**
   - **کجا زندگی می‌کند؟** در **فضای هسته (Kernel Space)**.
   - **چیست؟** این "پرونده مدیریتی" نخ در سیستم‌عامل است.
   - **شامل چه چیزهایی است؟** اشاره‌گر به پردازه والد، اشاره‌گر به KTHREAD و آدرس شروع اجرای نخ.
1. **KTHREAD (Kernel Thread Block - بلاک نخ هسته):**
   - **کجا زندگی می‌کند؟** در **فضای هسته (Kernel Space)**.
   - **چیست؟** این "برگه تقلب" زمان‌بند (Scheduler) است. تمام اطلاعاتی که زمان‌بند برای مدیریت نخ نیاز دارد، اینجاست.
   - **شامل چه چیزهایی است؟** اطلاعات زمان‌بندی (مثل اولویت)، اطلاعات همگام‌سازی، **پشته سطح هسته (Kernel Stack)** و یک اشاره‌گر به TEB.

نکته طلایی (پشته‌ها): هر نخ در ویندوز **دو پشته** دارد:

1. **پشته کاربر (در TEB):** وقتی نخ در حال اجرای کد عادی برنامه (در مد کاربر) است، از این پشته استفاده می‌کند.
2. **پشته هسته (در KTHREAD):** وقتی نخ یک فراخوانی سیستمی (System Call) انجام می‌دهد و وارد "مد هسته" می‌شود، بلافاصله به این پشته سوئیچ می‌کند.

۴-۷-۲: نخ‌های لینوکس (Linux Threads)

فلسفه اصلی: "نخ همان پردازه سبک است" فلسفه لینوکس به کلی متفاوت است. لینوکس **هیچ تفاوتی** بین پردازه و نخ قائل نمی‌شود!

- از دید هسته لینوکس، همه‌چیز "وظیفه" **(Task)** نامیده می‌شود.
- لینوکس به جای "نخ"، مفهومی به نام "پردازه سبک‌وزن" (LWP) را دارد.

جادوی `clone()` (سوال ۱۰۰٪ امتحانی) در فصل ۳ دیدیم که لینوکس با `fork()` یک پردازه جدید می‌سازد (که همه‌چیز را کپی می‌کند). لینوکس یک فراخوانی سیستمی قدرتمندتر به نام `clone()` هم دارد.

- `clone()` به شما اجازه می‌دهد مشخص کنید که "فرزند" چه چیزهایی را باید با "والد" **به اشتراک بگذارد**.
- این کار با "پرچم‌ها" (Flags) انجام می‌شود:
  - `CLONE_VM`: می‌گوید "فضای حافظه را اشتراک بگذار" (کپی نکن).
  - `CLONE_FILES`: می‌گوید "فایل‌های باز را اشتراک بگذار".
  - `CLONE_SIGHAND`: می‌گوید "مدیریت سیگنال‌ها را اشتراک بگذار".
  - و...

تفاوت `fork()` و `thread` در لینوکس:

1. **ساخت پردازه (مثل `fork()`):**
   - `clone()` را بدون **هیچ** پرچم اشتراک‌گذاری صدا می‌زنیم.
   - نتیجه: یک "وظیفه" (Task) جدید که **هیچ چیز** مشترکی با والد ندارد (به جز کپی اولیه). این یک **پردازه** است.
1. **ساخت نخ (Thread):**
   - `clone()` را با ***تمام*** پرچم‌های اشتراک‌گذاری (مثل `CLONE_VM`, `CLONE_FILES` و ...) صدا می‌زنیم.
   - نتیجه: یک "وظیفه" (Task) جدید که ***همه***‌چیز (حافظه، فایل‌ها، ...) را با والد اشتراک می‌گذارد. این یک **نخ** است.

ساختمان داده: `struct task_struct`

- در لینوکس، هر "وظیفه" (چه پردازه باشد چه نخ) یک ساختمان داده به نام `task_struct` در هسته دارد.
- این ساختار، خودش داده‌ها را نگه نمی‌دارد، بلکه ***اشاره‌گرهایی*** به ساختمان داده‌های دیگر (مثل لیست فایل‌های باز، اطلاعات حافظه و...) دارد.
- **وقتی `fork()` می‌کنید:** هسته از تمام آن ساختمان داده‌ها **کپی** می‌گیرد و اشاره‌گرهای `task_struct` جدید را به کپی‌ها وصل می‌کند.
- **وقتی `clone()` (برای نخ) می‌کنید:** هسته ***فقط*** `task_struct` را می‌سازد و اشاره‌گرهای آن را به ساختمان داده‌های ***والد*** وصل می‌کند (هیچ کپی‌ای انجام نمی‌شود).

نتیجه‌گیری مقایسه‌ای:

- **ویندوز:** نخ و پردازه دو موجود *کاملاً متفاوت* هستند. نخ در مدل یک-به-یک و با ساختارهای داده‌ای سه‌گانه (TEB, ETHREAD, KTHREAD) پیاده‌سازی می‌شود.
- **لینوکس:** نخ و پردازه *یکی* هستند (هر دو "Task" هستند). تفاوت آن‌ها فقط در **میزان اشتراک‌گذاری منابع** است که توسط پرچم‌های `clone()` تعیین می‌شود.